Aula 12 – Princípios SOLID com C#

Programação Orientada a Objetos
Professor: Eduardo Luan Pilonetto

Agenda da Aula

- Introdução aos princípios SOLID
- S – Single Responsibility Principle
- O – Open/Closed Principle
- L – Liskov Substitution Principle
- I – Interface Segregation Principle
- D – Dependency Inversion Principle
- Boas práticas em C#
- Exercícios práticos e revisão final

O que é SOLID?

- Conjunto de 5 princípios de design orientado a objetos
- Criados para melhorar manutenção e escalabilidade
- Facilitam testes, reutilização e legibilidade
- Muito utilizados em aplicações C# e .NET

Por que usar SOLID?

- Reduz acoplamento entre classes
- Facilita evolução do sistema
- Evita duplicação de código
- Melhora organização arquitetural

SOLID e C#

- C# oferece suporte completo aos princípios SOLID
- Interfaces e abstrações ajudam na aplicação
- Injeção de dependência é nativa no .NET
- Boas práticas favorecem APIs e sistemas corporativos

S – Responsabilidade Única (SRP)

- Uma classe deve ter apenas um motivo para mudar
- Cada classe deve possuir uma responsabilidade clara
- Separação de responsabilidades reduz complexidade
- Facilita manutenção e testes

```
public class RelatorioService
{
    public string GerarRelatorio() => "Relatório";
}

public class RelatorioEmail
{
    public void Enviar(string relatorio)
    {
        Console.WriteLine("Enviando relatório...");
    }
}
```

Problema sem SRP

- Classes gigantes acumulam muitas funções
- Mudanças em uma área afetam outras
- Baixa reutilização
- Código difícil de testar

```
public class Funcionario
{
    public void Salvar() {}
    public void GerarRelatorio() {}
    public void EnviarEmail() {}
}
```

O – Aberto/Fechado (OCP)

- Classes devem estar abertas para extensão
- Mas fechadas para modificação
- Novos comportamentos devem ser adicionados sem alterar código existente
- Uso comum: herança e interfaces

```
public interface IDesconto
{
    double Calcular(double valor);
}
```

Exemplo de OCP

- Adicionar novos descontos sem alterar regras existentes
- Cada estratégia possui sua própria implementação
- Sistema fica extensível

```
public class DescontoNatal : IDesconto
{
    public double Calcular(double
valor) => valor * 0.9;
}
```

L – Substituição de Liskov (LSP)

```
public abstract class Ave
{
    public abstract void Voar();
}
```

- Objetos derivados devem substituir objetos base sem erros
- Herança deve respeitar comportamento esperado
- Evitar sobrescrever regras incompatíveis

Violação do LSP

- Nem toda ave voa
- Pinguim herdando método Voar gera inconsistência
- Hierarquias devem representar comportamentos válidos

```
public class Pinguim : Ave
{
    public override void Voar()
    {
        throw new
Exception("Pinguins não voam");
    }
}
```

I – Segregação de Interface (ISP)

- Interfaces devem ser específicas
- Evitar interfaces gigantes
- Clientes não devem depender de métodos que não usam

```
public interface IImpressora
{
    void Imprimir();
}
```

Exemplo ISP

- Separação de impressão e scanner
- Dispositivos implementam apenas o necessário
- Menor acoplamento

```
public interface IScanner
{
    void Escanear();
}
```

D – Inversão de Dependência (DIP)

- Dependa de abstrações e não de implementações
- Classes de alto nível não devem depender de baixo nível
- Uso comum com injeção de dependência

```
public interface ILogger
{
    void Log(string mensagem);
}
```

Exemplo DIP

- Serviço depende de ILogger
- Permite trocar implementação sem alterar serviço
- Facilita testes unitários

```
public class PedidoService
{
    private readonly ILogger _logger;

    public PedidoService(ILogger logger)
    {
        _logger = logger;
    }
}
```

Resumo Final

- SOLID melhora manutenção e escalabilidade
- Aplicar princípios reduz acoplamento
- C# possui ótimos recursos para SOLID
- Boas práticas aumentam qualidade do software